- The log level of a message indicates how important it is and ranges from `Trace` to `Critical`. Typically, you'll create many low-importance log messages and a few high-importance log messages.
- You specify the log level of a log by using the appropriate extension method of `ILogger` to create your log. To write an `Information` level log, use `ILogger.LogInformation(message)`.
- The log category indicates which component created the log. It is typically set to the fully qualified name of the class creating the log, but you can set it to any string if you wish. `ILogger<T>` will have a log category of `T`.
- You can format messages with placeholder values, similar to the `string.Format` method, but with meaningful names for the parameters. Calling `logger.LogInfo ("Loading Recipe with id {RecipeId}", 1234)` would create a log reading `"Loading Recipe with id 1234"`, but it would also capture the value `RecipeId=1234`. This structured logging makes analyzing log messages much easier.
- ASP.NET Core includes many logging providers out of the box, including the console, debug, EventLog, and EventSource providers. Alternatively, you can add third-party logging providers.